# PROPOSAL ĐỒ ÁN

## KIẾN TRÚC & TỔ CHỨC DỮ LIỆU

# Phương Pháp Hình Thức Trong CNPM

Formal Methods Research Platform — Proposal Kiến Trúc & Dữ Liệu

Phiên bản 1.0

## Contents


Yêu cầu chung ..... 2
A. Nội dung Lý thuyết (Theoretical Foundation) ..... 2
B. Nền tảng Kỹ thuật & Công cụ (Tools & Technologies) ..... 2
C. Hướng dẫn Phương pháp làm (Methodology) ..... 2
D. Nội dung Thực nghiệm (Experimental Case Studies) ..... 3
E. Cấu trúc gợi ý cho báo cáo đồ án ..... 3
Giới thiệu về Alloy ..... 3
Ví dụ "Hệ thống mượn sách thư viện" ..... 6
Ví dụ "Hệ thống Đăng ký Học phần" ..... 7
1. Tổng Quan & Định Hướng ..... 10
2. Tổ Chức Dữ Liệu ..... 10
2.1 Tập Dữ Liệu Người Dùng & Nghiên Cứu Viên ..... 10
2.2 Tập Dữ Liệu Đặc Tả Hình Thức (Specifications) ..... 11
2.3 Dữ Liệu Kết Quả Verification ..... 11
2.4 Metadata & Knowledge Base ..... 12
3. Tính Năng ..... 12
3.1 Interactive Spec Editor & Feedback ..... 12
3.2 AI-Assisted Formal Specification ..... 12
3.3 Learning Management ..... 13
4. Dữ Liệu Thực ..... 13

# Yêu cầu chung

## A. Nội dung Lý thuyết (Theoretical Foundation)

Trong phần này, bạn cần làm rõ tại sao chúng ta cần toán học để xây dựng phần mềm.

- **Khái niệm:** Phương pháp hình thức là việc sử dụng các kỹ thuật dựa trên toán học (logic, tập hợp, đồ thị) để đặc tả, phát triển và xác minh các hệ thống phần mềm và phần cứng.
- **Các trụ cột chính:**
    - **Formal Specification (Đặc tả hình thức):** Dùng ngôn ngữ toán học để mô tả hệ thống *phải làm gì* (What) thay vì *làm như thế nào* (How). Ví dụ: Ngôn ngữ Z, B, VDM.
    - **Formal Verification (Xác minh hình thức):** Chứng minh tính đúng đắn của thiết kế so với đặc tả.
        - *Model Checking:* Kiểm tra tất cả các trạng thái có thể có của hệ thống (ví dụ: SPIN, NuSMV).
        - *Theorem Proving:* Sử dụng các quy tắc logic để chứng minh các thuộc tính (ví dụ: Coq, Isabelle/HOL).
- **Vị trí trong SDLC:** Giải thích phương pháp này áp dụng ở giai đoạn nào (thường là Đặc tả và Thiết kế) và nó giúp giảm thiểu lỗi từ sớm như thế nào.

---

## B. Nền tảng Kỹ thuật & Công cụ (Tools & Technologies)

Để đồ án mang tính thực tiễn, bạn nên chọn một hoặc hai công cụ phổ biến để nghiên cứu sâu:

- **Alloy:** Một ngôn ngữ đặc tả nhẹ (lightweight) dựa trên logic bậc nhất, rất phù hợp cho sinh viên vì nó có giao diện trực quan và dễ học.
- **TLA+:** Được sử dụng tại Amazon và Microsoft để kiểm tra các hệ thống phân tán phức tạp. Công cụ này rất mạnh để phát hiện các lỗi "race condition" hoặc "deadlock".
- **SPIN:** Chuyên dùng để kiểm tra các giao thức truyền thông và hệ thống đa luồng thông qua ngôn ngữ Promela.
- **Z3 Prover:** Một bộ giải SMT (Satisfiability Modulo Theories) cực mạnh từ Microsoft Research, thường dùng làm "động cơ" bên dưới cho nhiều công cụ khác.

---

## C. Hướng dẫn Phương pháp làm (Methodology)

Bạn nên triển khai đồ án theo các bước sau:

1. **Chọn bài toán thực tế:** Đừng cố gắng áp dụng phương pháp hình thức cho toàn bộ một hệ thống khổng lồ. Hãy chọn một module nhỏ nhưng phức tạp hoặc quan trọng.
2. **Xây dựng mô hình (Modeling):** Chuyển đổi yêu cầu từ ngôn ngữ tự nhiên sang ngôn ngữ hình thức (ví dụ: viết mã Alloy hoặc TLA+).

3. **Xác định thuộc tính (Properties):** Định nghĩa các tính chất mà hệ thống *bắt buộc* phải có.
    - *Safety:* Những điều xấu không bao giờ xảy ra (ví dụ: hai tàu hỏa không bao giờ ở trên cùng một đoạn ray).
    - *Liveness:* Những điều tốt cuối cùng sẽ xảy ra (ví dụ: yêu cầu của người dùng cuối cùng sẽ được đáp ứng).
4. **Kiểm tra và Chỉnh sửa (Verification & Iteration):** Chạy công cụ kiểm thử. Nếu công cụ tìm ra "Counter-example" (ví dụ phản chứng), bạn phải sửa lại thiết kế của mình.

---

## D. Nội dung Thực nghiệm (Experimental Case Studies)

Ví dụ một số hệ thống:

- **Hệ thống đèn giao thông:** Mô hình hóa các trạng thái đèn và chứng minh rằng không bao giờ có trường hợp hai hướng cùng xanh một lúc.
- **Hệ thống ATM/Ngân hàng:** Đặc tả các giao dịch rút tiền, chuyển tiền để đảm bảo tính toàn vẹn dữ liệu (số dư không bao giờ âm vô lý).
- **Giao thức đồng thuận (Consensus Protocol):** Mô hình hóa một thuật toán đơn giản (như 2-Phase Commit) để xem nó hoạt động thế nào khi có sự cố mạng.
- **Smart Contract:** Sử dụng phương pháp hình thức để kiểm tra lỗi bảo mật trong các hợp đồng thông minh trên Blockchain.

---

## E. Cấu trúc gợi ý cho báo cáo đồ án

| Chương | Nội dung chính |
|---|---|
| **Chương 1: Mở đầu** | Lý do chọn đề tài, mục tiêu và phạm vi nghiên cứu. |
| **Chương 2: Cơ sở lý thuyết** | Logic toán học, các phương pháp Model Checking, Theorem Proving. |
| **Chương 3: Công cụ đặc tả** | Giới thiệu chi tiết về công cụ bạn chọn (ví dụ: TLA+ hoặc Alloy). |
| **Chương 4: Thực hiện thực nghiệm** | Mô tả bài toán, mã nguồn đặc tả, các kịch bản lỗi phát hiện được. |
| **Chương 5: Đánh giá & Kết luận** | So sánh với phương pháp thử nghiệm (Testing) truyền thống, ưu và nhược điểm. |

---

**N**ên bắt đầu với công cụ **Alloy Analyzer** vì nó có khả năng xuất ra sơ đồ hình ảnh trực quan.

---

## Giới thiệu về Alloy

Alloy là một ngôn ngữ đặc tả dựa trên **Logic bậc nhất (First-order logic)** và **Logic quan hệ (Relational logic)**.

- **Đặc điểm:** Nó không dùng để lập trình tạo ra phần mềm chạy được, mà dùng để xây dựng "mô hình" của phần mềm để kiểm tra xem thiết kế có bị lỗi logic không.

- **Alloy Analyzer:** Là công cụ đi kèm giúp bạn "vẽ" ra các kịch bản của hệ thống và tự động tìm lỗi.

---

**Các thành phần cơ bản trong ngôn ngữ Alloy**
Bạn cần nắm vững 4 từ khóa "xương sống" sau:
**a. sig (Signatures): Định nghĩa đối tượng**
Dùng để khai báo các tập hợp thực thể trong hệ thống.

- sig Book {}: Khai báo tập hợp tất cả các cuốn sách.
- sig Member { borrows: set Book }: Khai báo tập hợp thành viên, trong đó mỗi thành viên có một quan hệ borrows tới một tập hợp (set) sách.

**b. fact (Facts): Các quy tắc bắt buộc**
Đây là những ràng buộc luôn luôn đúng trong hệ thống. Nếu thực tế vi phạm fact, hệ thống đó không tồn tại.

- *Ví dụ:* Một cuốn sách không thể ở hai nơi cùng lúc.

**c. pred (Predicates): Các hành động hoặc kịch bản**
Dùng để mô tả các thao tác thay đổi trạng thái hệ thống (như mượn sách, trả sách) hoặc định nghĩa một trạng thái cụ thể nào đó.
**d. assert (Assertions) & check: Kiểm tra tính đúng đắn**
Bạn đưa ra một giả thuyết: "Tôi tin rằng với các fact trên thì lỗi X sẽ không bao giờ xảy ra". Lệnh check sẽ bảo Alloy đi tìm bằng chứng phản bác (counter-example).

---

**Hướng dẫn chi tiết bài toán "Quản lý Thư viện"**
Hãy cùng viết mã Alloy hoàn chỉnh và phân tích từng dòng để bạn đưa vào đồ án:
Đoạn mã

```
/* 1. KHAI BÁO CÁC TẬP HỢP */
sig Book {}

sig Member {
   // Mỗi Member mượn một tập hợp Book
   borrows: set Book
}

/* 2. CÁC RÀNG BUỘC LOGIC (FACT) */
fact NoSharedBooks {
   // Với mọi cuốn sách b, tối đa chỉ có một thành viên mượn nó
   // 'lone' nghĩa là: 0 hoặc 1 (tối đa là 1)
   all b: Book | lone m: Member | b in m.borrows
}

/* 3. MÔ TẢ HÀNH ĐỘNG (PREDICATE) */
// Kịch bản mượn sách thành công
pred borrowBook [m: Member, b: Book] {
   // Điều kiện cần: Sách b chưa có ai mượn
   no m_other: Member | b in m_other.borrows
   // Kết quả: Sau khi mượn, b phải nằm trong tập borrows của m
   // (Trong mô hình tĩnh đơn giản, ta mô tả trạng thái mượn)
   b in m.borrows
}

/* 4. KIỂM TRA LỖI LOGIC (ASSERTION) */
// Giả thuyết: "Một cuốn sách không bao giờ thuộc về 2 người"
assert SafetyProperty {
   all b: Book | all m1, m2: Member |
```

```
    (b in m1.borrows and b in m2.borrows) => m1 = m2
}

// Lệnh thực thi: Kiểm tra giả thuyết trên trong phạm vi 5 đối tượng
check SafetyProperty for 5
```

---

**Cách sử dụng công cụ Alloy Analyzer (Thực nghiệm)**

1. **Cài đặt:** Tải file .jar từ AlloyTools.org (Yêu cầu máy có Java).
2. **Viết Code:** Dán đoạn mã trên vào trình soạn thảo.
3. **Chạy (Execute):** Nhấn nút **Execute**.
4. **Xem kết quả:**
    - Nếu dòng chữ **"No counterexample found"** hiện ra (màu xanh): Thiết kế của bạn an toàn.
    - Nếu hiện **"Counterexample found"** (màu đỏ): Có lỗi! Nhấn vào nút **Show** để xem sơ đồ.

---

**Cách đưa vào đồ án**

Để phần thực nghiệm chuyên nghiệp, bạn nên làm theo các bước:

- **Bước 1: Giả định sai.** Hãy thử xóa phần fact NoSharedBooks đi và chạy check. Alloy sẽ hiện ra một sơ đồ cực kỳ trực quan cho thấy 2 Member cùng mượn 1 Book. Chụp ảnh màn hình này đưa vào đồ án và ghi chú: *"Phát hiện lỗi logic: Hệ thống cho phép mượn trùng"*.
- **Bước 2: Sửa lỗi.** Thêm lại fact và chứng minh rằng sau khi thêm ràng buộc toán học, lỗi không còn nữa.
- **Bước 3: Mở rộng tính năng.** Thêm một thuộc tính mới:
    - *Yêu cầu:* "Mỗi người chỉ được mượn tối đa 3 cuốn".
    - *Mã Alloy:* all m: Member | #m.borrows <= 3 (Dấu # là đếm số lượng phần tử).

---

**Các toán tử logic hay dùng (Cần ghi nhớ)**

Trong báo cáo, bạn cần giải thích các ký hiệu này:

- all: Với mọi (Universal quantifier).
- some: Tồn tại ít nhất một.
- no: Không có cái nào.
- lone: Tối đa một (0 hoặc 1).
- one: Duy nhất một.
- in: Thuộc tập hợp / Quan hệ.
- #: Lấy số lượng (Cardinality).
- =>: Nếu... thì... (Implication).

Tóm lại, để đồ án về Alloy của bạn đạt kết quả tốt nhất, hãy tập trung vào:

1. **Mô hình hóa:** Chuyển đổi các quy tắc kinh doanh (Business Rules) thành các fact.

2. **Phát hiện lỗi:** Sử dụng lệnh check để tìm ra các trường hợp mà thiết kế ban đầu có thể bỏ sót (Counter-examples).
3. **Trực quan hóa:** Sử dụng sơ đồ từ Alloy Analyzer để minh họa sinh động cho phần thực nghiệm.

---

## Ví dụ "Hệ thống mượn sách thư viện"

Sử dụng công cụ **Alloy**.

**Bước 1: Phát biểu bài toán (Ngôn ngữ tự nhiên)**
Chúng ta cần thiết kế một hệ thống quản lý sách với các quy tắc (ràng buộc) sau:
1. Có hai đối tượng chính: **Sách (Book)** và **Thành viên (Member)**.
2. Một cuốn sách tại một thời điểm chỉ có thể ở một trong hai trạng thái: nằm trong thư viện hoặc đang được một thành viên mượn.
3. Một thành viên có thể mượn nhiều cuốn sách.
4. **Thuộc tính quan trọng (Property):** Không bao giờ có chuyện một cuốn sách được mượn bởi hai người khác nhau cùng lúc.

---

**Bước 2: Chuyển đổi sang Lý thuyết hình thức**
Chúng ta sử dụng logic tập hợp và quan hệ toán học để mô tả:
- Gọi B là tập hợp các cuốn sách.
- Gọi M là tập hợp các thành viên.
- Gọi borrowed_by là một quan hệ (relation) từ B \rightarrow M.
- **Điều kiện:** Quan hệ này phải là một hàm số (function), vì mỗi phần tử của B chỉ có thể ánh xạ tới tối đa một phần tử của M.

---

**Bước 3: Triển khai trên nền tảng kỹ thuật (Alloy)**
Bạn có thể viết mã này vào Alloy Tools:
Đoạn mã
```
// 1. Định nghĩa các đối tượng
sig Book {}
sig Member {
   borrows: set Book // Một thành viên có thể mượn một tập hợp sách
}

// 2. Ràng buộc (Fact): Đảm bảo tính đúng đắn của hệ thống
fact NoDoubleBorrowing {
   // Với mọi cuốn sách b1, b2, nếu chúng được mượn bởi các thành viên khác nhau
   // thì b1 và b2 không thể là cùng một cuốn sách.
   all b: Book | lone m: Member | b in m.borrows
}

// 3. Hành động (Pred): Mô phỏng việc mượn sách
pred borrow [m: Member, b: Book] {
   // Điều kiện: Sách b phải chưa bị ai mượn
   no m2: Member | b in m2.borrows
   // Sau đó: Thêm b vào danh sách mượn của m
```

```
    m.borrows' = m.borrows + b
}

// 4. Kiểm tra (Assert): Chứng minh thuộc tính an toàn
assert OneBookOneOwner {
    all b: Book | #( {m: Member | b in m.borrows} ) <= 1
}

check OneBookOneOwner // Lệnh yêu cầu Alloy kiểm tra xem có sai số không
```

---

**Bước 4: Thực nghiệm và Kiểm chứng (Verification)**
Khi bạn nhấn nút **Execute** trong Alloy:

1. **Nếu thiết kế đúng:** Công cụ sẽ báo "No counter-example found". Nghĩa là logic của bạn về mặt toán học là không thể bị phá vỡ trong phạm vi kiểm tra.
2. **Nếu thiết kế sai (Ví dụ: Bạn bỏ phần fact đi):** Alloy sẽ tạo ra một **Counter-example**. Nó sẽ vẽ ra một sơ đồ minh họa: "Đây, tôi tìm thấy trường hợp Thành viên A và Thành viên B cùng cầm cuốn sách C. Hệ thống của bạn đã lỗi!".

---

**Bước 5: Phân tích kết quả trong đồ án**
Trong báo cáo đồ án, bạn sẽ đưa ra các kết luận từ thực nghiệm này:

- **Ưu điểm:** Khác với Unit Testing (chỉ thử một vài trường hợp), phương pháp hình thức đã quét **tất cả** các trạng thái có thể xảy ra của quan hệ B \rightarrow M.
- **Giá trị:** Việc phát hiện lỗi "mượn trùng" ngay từ khâu thiết kế (trước khi viết code Java/Python) giúp tiết kiệm 90% chi phí sửa lỗi sau này.

---

**Triển khai**

1. **Cách cài đặt Alloy Analyzer** trên máy tính.
2. **Mở rộng bài toán:** Thêm quy tắc "Thành viên chỉ được mượn tối đa 3 cuốn sách" hoặc "Sách bị mất (Lost)".
3. **Viết chương 4 (Thực nghiệm)** cho báo cáo dựa trên ví dụ này.

---

## Ví dụ "Hệ thống Đăng ký Học phần".

Bài toán này phức tạp hơn bài toán thư viện một chút vì nó có mối quan hệ bắc cầu (tiên quyết) và các điều kiện ràng buộc chồng chéo.

**1. Phân tích bài toán (Scenario)**
Trong hệ thống đăng ký học phần, chúng ta có các thực thể và quy tắc sau:

- **Sinh viên (Student)** và **Môn học (Course)**.
- **Môn tiên quyết (Prerequisite):** Một môn học có thể có tối đa một môn tiên quyết (để đơn giản hóa).
- **Bảng điểm (Transcript):** Lưu lại những môn sinh viên đã vượt qua.
- **Đăng ký (Registration):** Những môn sinh viên đang đăng ký học.
- **Quy tắc an toàn (Safety Property):** Sinh viên **không được phép** đăng ký một môn học nếu họ chưa vượt qua môn tiên quyết của môn đó.

---

**2. Mô hình hóa hình thức (Formal Modeling)**
Trong báo cáo, bạn có thể diễn giải bằng toán học như sau:

- Gọi C là tập hợp môn học. Quan hệ tiên quyết là một hàm pre: C → C ∪ (∅).
- Gọi S là tập hợp sinh viên.
- Quan hệ đã học: Passed ⊆ S × C.
- Quan hệ đang đăng ký: Enrolled ⊆ S \× C.
- **Thuộc tính cần chứng minh:** ∀ s ∈ S, ∀ c ∈ C: (s, c) ∈ Enrolled → (pre(c) ≠ ∅ → (s, pre(c)) ∈ Passed).

---

**3. Triển khai mã Alloy**
Bạn có thể đưa đoạn mã này vào phần thực nghiệm của đồ án:
Đoạn mã

```
// 1. Định nghĩa thực thể
sig Student {}

sig Course {
   prereq: lone Course // Một môn có tối đa (lone) một môn tiên quyết
}

// 2. Trạng thái hệ thống
one sig University {
   passed: Student -> Course,    // Sinh viên đã đậu môn nào
   enrolled: Student -> Course  // Sinh viên đang đăng ký môn nào
}

// 3. Các ràng buộc logic (Facts)
fact NoSelfPrereq {
   // Một môn không thể là môn tiên quyết của chính nó (tránh vòng lặp)
   no c: Course | c in c.^prereq // ^ là toán học bao đóng bắc cầu (transitive closure)
}

fact ValidRegistration {
   // Quy tắc: Muốn đăng ký môn c, phải pass môn prereq của c
   all s: Student, c: Course |
      c in s.(University.enrolled) => (some p: c.prereq | p in s.(University.passed) or no c.prereq)
}

// 4. Kiểm tra giả thuyết (Assertion)
assert AcademicIntegrity {
   // Giả thuyết: Không sinh viên nào đang học một môn mà chưa pass môn tiên quyết
   all s: Student, c: Course |
      (c in s.(University.enrolled) and some c.prereq) => c.prereq in s.(University.passed)
}

// Lệnh thực thi
check AcademicIntegrity for 3 but 5 Course
```

---

**4. Phân tích và đưa vào đồ án**
**Cách khai thác để viết chương Thực nghiệm:**

1. **Chạy thử nghiệm thành công:** Chạy lệnh check, Alloy sẽ báo "No counter-example". Điều này chứng minh thiết kế logic của bạn về đăng ký môn học là hoàn hảo dưới các ràng buộc đã đặt ra.
2. **Tạo lỗi giả định (Fault Injection):** * Hãy thử xóa bỏ dòng fact ValidRegistration.
    - Khi đó, Alloy sẽ tìm ra một **Counter-example**: Một sinh viên đang học môn "Cấu trúc dữ liệu" nhưng trong bảng điểm (passed) lại trống rỗng, trong khi "Tin học đại cương" là môn tiên quyết.
    - **Hình ảnh minh họa:** Chụp lại sơ đồ các mũi tên (Student -> Course) mà Alloy xuất ra. Đây chính là bằng chứng cho thấy nếu thiếu ràng buộc hình thức, phần mềm sẽ để lọt lưới các trường hợp sai trái.

**Điểm nhấn kỹ thuật cho đồ án:**

- **Toán tử ^ (Transitive Closure):** Giải thích trong báo cáo rằng toán tử này giúp kiểm tra toàn bộ chuỗi môn tiên quyết (ví dụ: A là tiên quyết của B, B là tiên quyết của C thì A gián tiếp là tiên quyết của C). Đây là thứ mà unit test thông thường rất khó kiểm tra hết.
- **Tính toàn vẹn (Integrity):** Nhấn mạnh rằng phương pháp hình thức giúp đảm bảo "Business Logic" của hệ thống đào tạo luôn đúng 100% trước khi viết bất kỳ dòng mã Java/C# nào.

---

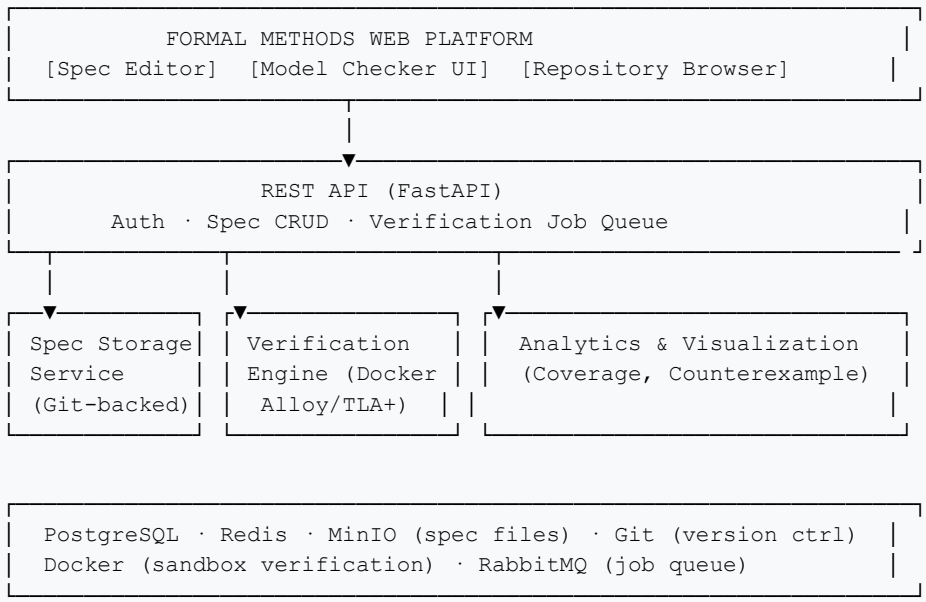
**5. Một số gợi ý mở rộng cho đồ án**

- **Ràng buộc số tín chỉ:** all s: Student | #s.(University.enrolled) <= 6 (Mỗi kỳ không đăng ký quá 6 môn).
- **Trạng thái tốt nghiệp:** Định nghĩa một pred isEligibleToGraduate [s: Student] nếu sinh viên đã passed tất cả các môn bắt buộc.

# 1. Tổng Quan & Định Hướng

Đề tài nghiên cứu phương pháp hình thức. Đề xuất xây dựng Formal Specification Repository & Verification Management System (FSRVMS) — nền tảng quản lý, lưu trữ, chia sẻ và phân tích các đặc tả hình thức (Alloy, TLA+, Z) cùng kết quả verification.

**Kiến Trúc — Formal Methods Research Platform**




# 2. Tổ Chức Dữ Liệu

## 2.1 Tập Dữ Liệu Người Dùng & Nghiên Cứu Viên

**Schema — Users & Projects**

```
TABLE: users
  user_id    UUID PK
  email      VARCHAR(255) UNIQUE
  full_name  VARCHAR(200)
  role       ENUM(student, researcher, lecturer, admin)
  affiliation VARCHAR(300)  -- trường/công ty
  expertise  TEXT[]  -- ['Alloy','TLA+','Z language','Model Checking']
  created_at TIMESTAMPTZ

TABLE: research_projects
  project_id  UUID PK
  name        VARCHAR(300)
  description TEXT
  owner_id    UUID FK → users
  visibility  ENUM(private, team, public)
  domain      VARCHAR(100)  -- 'Distributed Systems','Security Protocol'
  created_at  TIMESTAMPTZ

TABLE: project_members
  project_id UUID FK, user_id UUID FK
  role       ENUM(owner, editor, viewer)
  PRIMARY KEY (project_id, user_id)
```

## 2.2 Tập Dữ Liệu Đặc Tả Hình Thức (Specifications)

### Schema — Formal Specifications

```
TABLE: specifications
  spec_id       UUID PK
  project_id    UUID FK
  name          VARCHAR(300)
  tool          ENUM(alloy, tla_plus, z_language, b_method, vdm, spin)
  domain        ENUM(safety, security, concurrency, data_integrity, other)
  description   TEXT
  current_version UUID FK → spec_versions
  created_by    UUID FK → users
  created_at    TIMESTAMPTZ

TABLE: spec_versions  -- Git-like version tracking
  version_id    UUID PK
  spec_id       UUID FK
  version_tag   VARCHAR(20)  -- 'v1.0', 'v1.1-draft'
  file_content  TEXT  -- nội dung spec
  file_hash     VARCHAR(64)  -- SHA-256 của content
  parent_version UUID FK → spec_versions
  commit_message TEXT
  created_by    UUID FK → users
  created_at    TIMESTAMPTZ

TABLE: formal_properties
  prop_id      UUID PK
  spec_id      UUID FK
  name         VARCHAR(200)  -- 'NoDeadlock','SafetyInvariant'
  type         ENUM(invariant, assertion, liveness, safety)
  expression   TEXT  -- biểu thức formal
  description  TEXT  -- giải thích ngôn ngữ tự nhiên
```

## 2.3 Dữ Liệu Kết Quả Verification

### Schema — Verification Results

```
TABLE: verification_jobs
  job_id        UUID PK
  spec_id       UUID FK
  version_id    UUID FK
  property_ids  UUID[]  -- properties cần check
  tool_config   JSONB  -- { scope: 5, bitwidth: 3, solver: 'minisat' }
  status        ENUM(queued, running, completed, failed, timeout)
  submitted_by  UUID FK → users
  submitted_at  TIMESTAMPTZ
  started_at    TIMESTAMPTZ
  finished_at   TIMESTAMPTZ
  duration_sec  INT

TABLE: verification_results
  result_id       UUID PK
  job_id          UUID FK
  property_id     UUID FK
  outcome         ENUM(satisfied, violated, unknown, timeout)
  states_explored INT  -- số trạng thái đã khám phá
  counterexample  JSONB  -- trace dẫn đến violation (if any)
  witness         JSONB  -- trace chứng minh satisfied (if any)
```

```
  memory_used_mb  INT
  raw_output      TEXT

TABLE: counterexample_traces  -- chi tiết trace
  trace_id    UUID PK
  result_id   UUID FK
  step_number INT
  state       JSONB  -- giá trị các biến ở bước này
  action      VARCHAR(200)
```

### 2.4 Metadata & Knowledge Base

**Schema — Metadata & Knowledge Base**

```
TABLE: spec_tags
  spec_id UUID FK, tag VARCHAR(100)

TABLE: spec_examples  -- thư viện ví dụ mẫu
  example_id  UUID PK
  tool        ENUM(alloy, tla_plus, ...)
  domain      VARCHAR(100)
  title       VARCHAR(300)
  description TEXT
  content     TEXT  -- spec code
  difficulty  ENUM(beginner, intermediate, advanced)
  references  TEXT[]  -- papers, books
  created_by  UUID FK

-- Elasticsearch index cho tìm kiếm spec
INDEX: specifications_search
{
  "spec_id": "uuid",
  "name": "Library Borrowing System",
  "tool": "alloy",
  "domain": "data_integrity",
  "tags": ["library","borrowing","alloy-tutorial"],
  "description_text": "... analyzed ...",
  "properties_count": 5,
  "verification_status": "all_satisfied"
}
```

## 3. Tính Năng

### 3.1 Interactive Spec Editor & Feedback

- Monaco Editor (VS Code engine) tích hợp trong browser
- Syntax highlighting + error detection cho Alloy, TLA+, Z
- One-click Submit to Verification Engine (sandboxed Docker container)
- Diff view: so sánh hai phiên bản spec, highlight thay đổi ảnh hưởng đến verification

### 3.2 AI-Assisted Formal Specification

- 'Viết Alloy model cho hệ thống mượn sách với tối đa 3 sách/người' → LLM generation
- Natural language → Formal property: 'Không bao giờ có 2 transaction cùng modify record' → assertion

- Counterexample explainer: AI giải thích trace phản ví dụ bằng ngôn ngữ tự nhiên

### 3.3 Learning Management

- Tutorial progression: hướng dẫn từng bước từ Alloy cơ bản đến advanced
- Exercise grader: tự động check correctness của bài tập sinh viên
- Peer review: chia sẻ spec để đồng nghiệp review và comment

## 4. Dữ Liệu Thực

| Loại dữ liệu | Target | Ghi chú |
|---|---|---|
| spec_examples | 200+ | Từ giáo trình, papers, Alloy community |
| verification_results | 5,000+ | Chạy verification cho tất cả examples |
| users | 200+ | Sinh viên khóa học Phương pháp hình thức |
| research_projects | 50+ | Projects mẫu theo domain |